

EXAMEN DE PROGRAMACIÓN – 1º GRADO SUPERIOR

Examen 2 · Arrays y Strings

Arrays unidimensionales: declaración, inicialización y recorrido

Arrays bidimensionales (matrices)

Ordenación y búsqueda (burbuja, selección, búsqueda binaria)

La clase String: inmutabilidad, métodos, comparación

Ejercicios con Scanner y procesamiento de texto

Asignatura	Programación (PRG)
Curso	1º Grado Superior — Desarrollo de Aplicaciones Web (DAW)
Duración	2 horas
Puntuación total	16 puntos
Convocatoria	Evaluación 1
Valoración	Arrays, matrices y Strings

Parte 1: Opción múltiple y Verdadero/Falso

5 puntos — 0,5 cada pregunta

1. Declaración correcta de un array de 5 enteros:

- a) `int [] nums = int[5];`
- b) `int[] nums = new int[5];`
- c) `int nums[]; nums = [5];`
- d) `int nums = new int[5];`

RESPUESTA / SOLUCIÓN

b) `int[] nums = new int[5];` — Crea un array de 5 enteros, inicializados a 0.

2. ¿Cuántos elementos hay en la matriz `int[][] m = {{1,2,3},{4,5}};`?

- a) 6
- b) 5
- c) 3
- d) 2

RESPUESTA / SOLUCIÓN

b) 5 — La primera fila tiene 3 elementos y la segunda 2: $3 + 2 = 5$.

3. Dado:

```
String a = "Hola";  
String b = new String("Hola");  
System.out.println(a == b);
```

- a) true
- b) false
- c) Error de compilación
- d) Hola

RESPUESTA / SOLUCIÓN

b) false — ``==`` compara referencias. ``new String(...)`` crea un nuevo objeto en el heap, distinto del literal del pool, por lo que las referencias son distintas aunque el contenido sea igual.

4. ¿Qué devuelve `"programacion".substring(3, 7)`?

- a) gram

- b) ogra
- c) grama
- d) ogram

RESPUESTA / SOLUCIÓN

a) gram — `substring(3,7)` devuelve los caracteres desde el índice 3 (inclusive) hasta el 7 (exclusivo): g,r,a,m = 'gram'.



5. Las cadenas String en Java son mutables, es decir, su contenido se puede cambiar una vez creadas.

RESPUESTA / SOLUCIÓN

Falso — La clase String es inmutable. Cada operación que parece 'modificarla' crea una nueva cadena.



6. Tras ejecutar `int[] a = {5,3,8,1}; Arrays.sort(a);` el array queda:

- a) {5,3,8,1}
- b) {1,3,5,8}
- c) {8,5,3,1}
- d) No se puede ordenar

RESPUESTA / SOLUCIÓN

b) {1,3,5,8} — `Arrays.sort` ordena ascendentemente.



7. La búsqueda binaria (`binarySearch`) requiere que el array esté:

- a) Al revés
- b) Sin importar el orden
- c) Ordenado ascendentemente
- d) Lleno de números únicos

RESPUESTA / SOLUCIÓN

c) Ordenado ascendentemente — La búsqueda binaria divide el intervalo, lo que solo es correcto si el array está ordenado.

8. ¿Cuál es la sintaxis correcta para recorrer un array con for-each?

- a) for (int i : numeros) { }
- b) for (int numeros[0]) { }
- c) for-each (int n) { }
- d) for (int i = numeros) { }

RESPUESTA / SOLUCIÓN

a) for (int i : numeros) { } — El for-each declara una variable y usa ':' seguido del array.

9. ¿Qué imprime este código?

```
String s = "Abc";  
System.out.println(s.concat("DEF") + s);
```

- a) AbcDEFAbc
- b) AbcDEF
- c) AbcAbcDEF
- d) abcDEF

RESPUESTA / SOLUCIÓN

a) AbcDEFAbc — concat devuelve una nueva cadena "AbcDEF"; como String es inmutable, s sigue siendo "Abc". Al concatenar queda "AbcDEFAbc".

10. Para comparar el contenido de dos Strings correctamente se usa:

- a) a == b
- b) a.equals(b)
- c) a.compare(b)
- d) String.equals(a,b)

RESPUESTA / SOLUCIÓN

b) `a.equals(b)` — Compara el contenido textual; ``==`` compara referencias.



Parte 2: Análisis y depuración

3 puntos

1. El siguiente código tiene 3 errores. Localízalos y explica cómo corregirlos. Además, indica qué se esperaba imprimir si funcionara. (1,5 pts)

```
public class ArraysBug {
    public static void main(String[] args) {
        int[] nums = {3, 1, 4, 1, 5};
        System.out.println(suma(nums));
        System.out.println(nums);
    }
    static int suma(int[] n) {
        int t = 0;
        for (int i = 0; i <= n.length; i++)
            t += n[i];
        return t;
    }
}
```

RESPUESTA / SOLUCIÓN

1. ``i <= n.length`` → fuera de límite; debe ser ``i < n.length``.
2. ``System.out.println(nums)`` → imprime la referencia [`l@...`, no el contenido. Debe ser ``Arrays.toString(nums)``.
3. (real) El método ``suma`` devolvería la suma correcta si se corrige el bucle. Con los datos, la suma esperada es 14.

-
2. Analiza y determina la salida exacta de este código (traza). (1,5 pts)

```
public class MatrizT {
    public static void main(String[] args) {
        int[][] m = {{1, 2}, {3, 4}, {5, 6}};
        System.out.println("F: " + m.length);
        System.out.println("C: " + m[0].length);
        System.out.println(m[2][1] - m[0][0]);
        for (int i = 0; i < m.length; i++)
            System.out.print(m[i][i%2] + " ");
    }
}
```

RESPUESTA / SOLUCIÓN

m es una matriz de 3 filas y 2 columnas.

Salida: F: 3, C: 2, luego ``m[2][1] - m[0][0] = 6 - 1 = 5``.

El bucle imprime `m[0][0]=1, m[1][1]=4, m[2][0]=5` → ``1 4 5``.

Parte 3: Ejercicios de programación

7 puntos

1. Pide 8 números por teclado y guárdalos en un array. Implementa métodos para: (1) contar cuántos son pares, (2) calcular el mayor y el menor, y (3) invertir el array. Muestra el resultado. (2,5 pts)

RESPUESTA / SOLUCIÓN

```
import java.util.Scanner;

public class ArraysUtils {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int[] nums = new int[8];

        for (int i = 0; i < nums.length; i++) stats: nums[i] = sc.nextInt(); // simplificado

        System.out.println("Pares: " + contarPares(nums));

        System.out.println("Mayor: " + mayor(nums) + " Menor: " + menor(nums));

        int[] inv = invertir(nums);

        System.out.println("Invertido: " + java.util.Arrays.toString(inv));

    }

    static int contarPares(int[] a) { int c=0; for (int v:a) if (v%2==0) c++; return c; }

    static int mayor(int[] a) { int m=a[0]; for (int v:a) if (v>m) m=v; return m; }

    static int menor(int[] a) { int m=a[0]; for (int v:a) if (v<m) m=v; return m; }

    static int[] invertir(int[] a) {

        int[] r = new int[a.length];

        for (int i = 0; i < a.length; i++) r[i] = a[a.length-1-i];

        return r;

    }

}
```

2. Una matriz de 3x3 representa una tabla de números. Escribe un programa que la rellene con los valores de la multiplicación $i*j$ (tabla de multiplicar) y que luego sume la diagonal principal y la diagonal secundaria. (2 pts)

RESPUESTA / SOLUCIÓN

```
public class MatrizDiag {  
    public static void main(String[] args) {  
        int[][] m = new int[3][3];  
        int diagP = 0, diagS = 0;  
        for (int i = 0; i < 3; i++)  
            for (int j = 0; j < 3; j++) {  
                m[i][j] = i * j;  
                if (i == j) diagP += m[i][j];  
                if (i + j == 2) diagS += m[i][j];  
            }  
        System.out.println("Diagonal principal: " + diagP);  
        System.out.println("Diagonal secundaria: " + diagS);  
    }  
}
```

-
3. Escribe un programa que, dada una cadena introducida por teclado, cuente cuántas vocales contiene, cuántas consonantes y cuántas palabras tiene (separadas por espacios). (2 ptos)

RESPUESTA / SOLUCIÓN

```
import java.util.Scanner;  
public class ContadorTexto {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        String t = sc.nextLine().toLowerCase();  
        int voc = 0, cons = 0;  
        for (char c : t.toCharArray()) {  
            if (c >= 'a' && c <= 'z') {  
                if ("aeiou".indexOf(c) != -1) voc++; else cons++;  
            }  
        }  
        String[] pal = t.trim().split("\\s+");  
        int n = (t.trim().isEmpty()) ? 0 : pal.length;  
        System.out.println("Vocales: " + voc + " Consonantes: " + cons + " Palabras: " + n);  
    }  
}
```

-
4. Escribe un programa que ordene un array usando el método de la burbuja y, tras

ordenarlo, busque un valor introducido por teclado mediante búsqueda binaria, mostrando su posición o -1 si no existe. (2,5 ptos)

RESPUESTA / SOLUCIÓN

```
import java.util.Scanner;
import java.util.Arrays;
public class OrdenaBusca {
    static void burbuja(int[] a) {
        for (int i = 0; i < a.length-1; i++)
            for (int j = 0; j < a.length-1-i; j++)
                if (a[j] > a[j+1]) { int t=a[j]; a[j]=a[j+1]; a[j+1]=t; }
    }
    static int binaria(int[] a, int valor) {
        int izq=0, der=a.length-1;
        while (izq <= der) {
            int med = (izq+der)/2;
            if (a[med]==valor) return med;
            if (a[med] < valor) izq = med+1; else der = med-1;
        }
        return -1;
    }
    public static void main(String[] args) {
        int[] arr = {9, 3, 7, 1, 5, 2, 8, 4};
        burbuja(arr);
        System.out.println(Arrays.toString(arr));

        Scanner sc = new Scanner(System.in);
        System.out.print("Buscar: ");
        System.out.println("Posición: " + binaria(arr, sc.nextInt()));
    }
}
```

Parte 4: Pregunta teórica

1 punto

1. ¿Por qué es importante que un array esté ordenado para aplicar búsqueda binaria? Explica el funcionamiento paso a paso y su complejidad en notación O grande.

RESPUESTA / SOLUCIÓN

La búsqueda binaria divide repetidamente el intervalo de búsqueda a la mitad comparando el elemento central. Esto solo funciona si el array está ordenado, porque permite descartar la mitad izquierda o derecha con una sola comparación.

Complejidad: $O(\log n)$ en el peor caso porque se reduce a la mitad en cada paso ($\log_2 n$ pasos). Es mucho más eficiente que la búsqueda lineal $O(n)$ para datos grandes.



Plantilla de respuestas

(Páginas en blanco para desarrollar las soluciones)